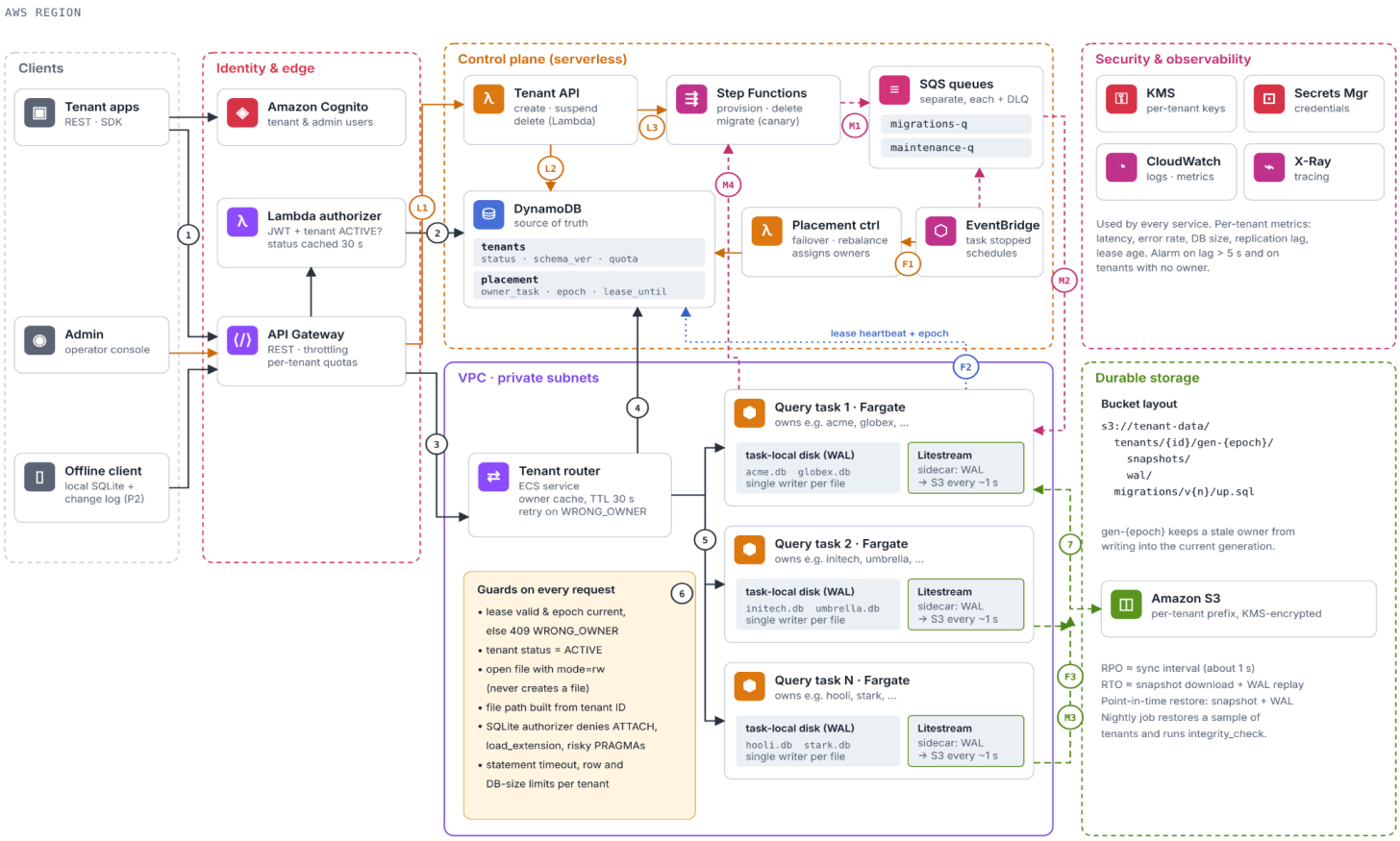
Scalable Multi-Tenant Database Service: Architecture

Cloud Computing & Big Data - Matt Boubin (mjb9353) · Khushi Agarwal (ka2653) · Vaishnavi Rathod (vsr6010) ·



Architecture Description

The service gives every tenant its own SQLite database. The central decision is **single-writer ownership**: each tenant's database lives on the local disk of exactly one Fargate task, and only that task may open it. SQLite's WAL mode relies on shared memory, which can't be coordinated across hosts, so two machines must never open the same file.

Request path (1–7)

1. Tenant apps call API Gateway, which applies per-tenant quotas and throttling.
2. A Lambda authorizer validates the Cognito JWT and confirms the tenant is ACTIVE in DynamoDB (cached 30 s).
3. API Gateway forwards the request through a VPC Link to the tenant router (ECS).
4. The router looks up the tenant's owner in the placement table (cached 30 s).
5. The router sends the query to the owning task. On 409 WRONG_OWNER, it refreshes its cache and retries.
6. The task checks its lease, epoch and tenant status, then opens the file read-write (never create) at a path built from the authenticated tenant ID. SQL runs under an authorizer that blocks ATTACH, extensions and risky PRAGMAs, with limits on statements, rows and size.
7. A Litestream sidecar streams the WAL to S3 about every second.

Offline clients (P2) use the same path: they push their local change log, pull newer changes, and conflicts resolve last-write-wins.

Tenant lifecycle (L1–L3). Admins call the Tenant API Lambda (L1), which writes the tenant record (L2) and starts a Step Functions workflow (L3). The workflow queues a provisioning job for the assigned owner task, which creates the empty database file. Explicit states

mean a partial failure shows as FAILED, not as a tenant missing its file.

Migrations (M1–M4). Step Functions puts jobs on migrations-q with a task token (M1). The owner task receives the job (M2), fetches migrations/v{n}/up.sql from S3 (M3), and reports the result back (M4), which enables canary batches and halt-on-failure. Owner tasks also consume maintenance-q (EventBridge schedules), so long migrations never delay maintenance.

Failover (F1–F3). Tasks heartbeat their lease and epoch to DynamoDB (F2). When EventBridge reports a stopped task or a lease expires (F1), the placement controller reassigns that task's tenants by load, with epoch + 1. The new owner restores the latest snapshot and replays the WAL from S3 (F3). The epoch is part of the S3 path, so a stale owner can't overwrite the current generation.

Security and observability. KMS encrypts each tenant's S3 prefix with its own key, and Secrets Manager holds credentials. CloudWatch and X-Ray track per-tenant latency, errors, DB size and replication lag, alarming when lag passes 5 s or a tenant has no owner. A nightly job test-restores a sample of tenants.

Design Decisions & Trade-offs

Durability vs. latency: Local disk makes commits fast, but a crash can lose about one second of writes (RPO ≈ 1 s), and the tenant is offline while it restores.
Single writer: Tasks can't corrupt each other's files, but one task caps each tenant's throughput, and Fargate's ephemeral storage caps database size.
Caching: The 30 s caches cut DynamoDB reads, but a suspension takes up to 30 s to apply, and routing is briefly stale until the 409 retry.
Isolation: Tenants on the same task share a process; tenants needing stronger isolation can get a dedicated task.
Per-tenant keys: Separate KMS keys isolate tenants and allow crypto-shredding on deletion, but cost about $1/month each; at large scale we would use one key with per-tenant encryption context.
Serverless control plane: Cheap and nothing to manage; cold starts only slow admin operations, not queries.